

Scripting en sistemas operativos

ADMINISTRACIÓN DE SISTEMAS OPERATIVOS

Que es un Script

Los **scripts** son conjuntos de instrucciones escritas en un lenguaje de programación que se utilizan para automatizar tareas, ejecutar funciones específicas o controlar el comportamiento de un programa, sistema o aplicación.

A diferencia de los programas complejos, los scripts suelen ser más sencillos, no requieren compilación (se ejecutan directamente) y están diseñados para resolver problemas concretos de manera eficiente

Características de un Script

- **Lenguaje Interpretado:** Se ejecuta directamente por un intérprete sin necesidad de compilación (ejemplo: Bash, Python, PowerShell, JavaScript, etc.).
- **Automatización:** Permite automatizar tareas del sistema, administración de servidores, generación de informes, entre otros.
- **Portabilidad:** Muchos scripts pueden ejecutarse en diferentes sistemas sin modificaciones, siempre que el intérprete esté disponible.
- **Facilidad de Edición:** Se puede modificar rápidamente con cualquier editor de texto sin necesidad de recompilación.
- **Uso en Desarrollo Web:** Lenguajes como JavaScript permiten manipular elementos del HTML y CSS para mejorar la experiencia del usuario.

Para qué se usan los Scripts?

- Automatización de tareas repetitivas en sistemas operativos.
- Administración de servidores y redes.
- Desarrollo de aplicaciones web.
- Manejo y procesamiento de datos.
- Instalación y configuración de software.

Tipos de Scripts

- **Scripts de Shell (Bash, PowerShell)**
 - Se usan para la administración de sistemas operativos.
- **Scripts en Python**
 - Usados en automatización, desarrollo web, análisis de datos, etc.
- **Scripts en JavaScript**
 - Ejecutados en navegadores para interacción web.
- **Scripts SQL**
 - Utilizados para consultas y manipulación de bases de datos.

Estructura de los Scripts

Variables

Las variables permiten almacenar valores y utilizarlos en diferentes partes del script.

Bash (Linux)

- No se usa \$ al asignar, pero sí al referenciar la variable.
- No es necesario declarar el tipo de variable.
- Se pueden definir variables de solo lectura con readonly.

Ejemplo:

bash

```
#!/bin/bash nombre="Juan" echo "Hola, $nombre"
```

PowerShell (Windows)

- Se usa \$ tanto para asignar como para referenciar.
- PowerShell detecta automáticamente el tipo de dato.

Ejemplo:

powershell

```
$nombre = "Juan" Write-Output "Hola, $nombre"
```

Estructura de los Scripts

Condicionales (if, elif, else)

Permiten evaluar condiciones y tomar decisiones.

Bash (Linux)

- Usa if, elif, else con test ([y] también sirven).
- Se necesita then después de if

Ejemplo:

```
#!/bin/bash
numero=10

if [ $numero -gt 5 ]; then
    echo "El número es mayor que 5"
elif [ $numero -eq 5 ]; then
    echo "El número es 5"
else
    echo "El número es menor que 5"
fi
```

```
(kali@DESKTOP-GNI2TT0)-[~]
$ #!/bin/bash
numero=10

if [ $numero -gt 5 ]; then
    echo "El número es mayor que 5"
elif [ $numero -eq 5 ]; then
    echo "El número es 5"
else
    echo "El número es menor que 5"
fi
El número es mayor que 5
```

Estructura de los Scripts

Condicionales (if, elif, else)

- Permiten evaluar condiciones y tomar decisiones.

PowerShell (Windows)

- Usa if, elseif, else.
- Las condiciones van entre () y los bloques dentro de {}.

Ejemplo:

```
$numero = 10
```

```
if ($numero -gt 5) {  
    Write-Output "El número es mayor que 5"  
} elseif ($numero -eq 5) {  
    Write-Output "El número es 5"  
} else {  
    Write-Output "El número es menor que 5"  
}
```

```
PS C:\Users\Federico> if ($numero -gt 5) {  
>>     Write-Output "El número es mayor que 5"  
>> } elseif ($numero -eq 5) {  
>>     Write-Output "El número es 5"  
>> } else {  
>>     Write-Output "El número es menor que 5"  
>> }  
El número es mayor que 5
```

Estructura de los Scripts

Bucles (for, while)

- Permiten repetir acciones múltiples veces.

Bash

```
for i in {1..5}; do  
    echo "Número: $i"  
done
```

```
(kali@DESKTOP-GNI2TT0)~  
$ #!/bin/bash  
for i in {1..5}; do  
    echo "Número: $i"  
done  
Número: 1  
Número: 2  
Número: 3  
Número: 4  
Número: 5
```

Bash

```
for color in rojo azul verde;  
do  
    echo "Color: $color"  
done
```

```
(kali@DESKTOP-GNI2TT0)~  
$ for color in rojo azul verde; do  
    echo "Color: $color"  
done  
Color: rojo  
Color: azul  
Color: verde
```

Estructura de los Scripts

PowerShell

```
for ($i=1; $i -le 5; $i++) {  
    Write-Output "Número: $i"  
}
```

```
PS C:\Users\Federico> for ($i=1; $i -le 5; $i++) {  
>>     Write-Output "Número: $i"  
>> }  
Número: 1  
Número: 2  
Número: 3  
Número: 4  
Número: 5
```

Estructura de los Scripts

Funciones

- Permiten definir bloques de código reutilizables.

- Bash

```
saludar() { echo "Hola,  
bienvenido" }
```

saludar # Llamada a la función

```
(kaliⓈ DESKTOP-GNI2TT0)-[~]  
$ saludar() {  
  echo "Hola, bienvenido"  
}
```

```
(kaliⓈ DESKTOP-GNI2TT0)-[~]  
$ saludar  
Hola, bienvenido
```

Powershell

```
function Saludar { Write-Output "Hola,  
bienvenido" }
```

Saludar # Llamada a la función

```
PS C:\Users\Federico> function Saludar { Write-Output "Hola, bienvenido" }  
PS C:\Users\Federico> saludar  
Hola, bienvenido  
PS C:\Users\Federico> |
```

Diferencias Claves entre Bash y PowerShell

Característica	Bash (Linux)	PowerShell (Windows)
Definir variable	<code>nombre="Juan"</code>	<code>\$nombre = "Juan"</code>
Referenciar variable	<code>\$nombre</code>	<code>\$nombre</code>
Condicional (if)	<code>if [\$x -eq 5]; then</code>	<code>if (\$x -eq 5) {}</code>
Bucle for	<code>for i in {1..5}; do</code>	<code>for (\$i=1; \$i -le 5; \$i++) {}</code>
Bucle while	<code>while [\$x -lt 5]; do</code>	<code>while (\$x -lt 5) {}</code>
Definir función	<code>funcion() {}</code>	<code>function Nombre {}</code>
Comentario	<code># Esto es un comentario</code>	<code># Esto es un comentario</code>

Creación y Depuración de Scripts en Windows Server y Linux

Selección del Lenguaje de Scripting

Sistema Operativo	Lenguajes de Scripting Comunes	Usos Frecuentes
Windows Server	- PowerShell (.ps1) - VBScript (.vbs) - Batch (.bat)	Administración del sistema, automatización de tareas, gestión de usuarios y archivos.
Linux	- Bash (.sh) - Python (.py) - Perl (.pl)	Administración del sistema, monitoreo, configuración de servidores, automatización.

Creación y Depuración de Scripts en Windows Server y Linux

Escritura del Script

- **Dividir el código en secciones lógicas:** Separar la inicialización, el procesamiento y la finalización.
- **Comentar el código:** Explicar cada parte con comentarios (# en Bash y PowerShell).
- **Seguir buenas prácticas:** Usar nombres de variables descriptivos y evitar la repetición de código.

Creación y Depuración de Scripts en Windows Server y Linux

Prueba del Script

Antes de ejecutar un script en un entorno de producción, es crucial probarlo y depurarlo.

Herramientas de Depuración en Linux (Bash)

- **Modo de depuración (-x):** Muestra cada comando ejecutado antes de ejecutarlo.
 - `bash -x script.sh`
- **Modo estricto (-e):** Detiene la ejecución si un comando falla.
 - `bash -e script.sh`

Creación y Depuración de Scripts en Windows Server y Linux

Herramientas de Depuración en Windows (PowerShell)

- Modo paso a paso (-Debug): Muestra detalles de ejecución.

```
Set-PSDebug -Trace 2
```

Try-Catch para capturar errores:

```
try {  
    Get-Content "C:\archivo_script.txt"  
} catch {  
    Write-Output "Error: No se encontró el archivo"}
```

Ejecucion de Scripts

Ejecutar un Script en Linux (Bash)

- **Dar permisos de ejecución** (solo la primera vez):
 - `chmod +x script.sh`
- **Ejecutar el script:**
 - `./script.sh`
- **Ejecutar con un intérprete específico:**
 - `bash script.sh`

Ejecutar un Script en Windows (PowerShell)

- **Permitir la ejecución de scripts** (si es necesario):
 - `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`
- **Ejecutar el script:**
 - `.\script.ps1`
- **Ejecutar desde PowerShell con ruta completa:**
 - `PowerShell -File "C:\ruta\script.ps1 "`

Automatizacion de Scripts

Programación Automática de Scripts

Para ejecutar scripts automáticamente en horarios programados:

Linux (Usando cron)

- **Abrir el editor de crontab:**
 - `crontab -e`
- **Agregar una tarea programada (ejecutar cada día a las 3:00 AM):**
 - `bash`
 - `0 3 * * * /ruta/script.sh`

Automatizacion de Scripts

Windows (Usando el Programador de Tareas)

Abrir "Programador de tareas" (taskschd.msc).

Crear una nueva tarea y seleccionar Ejecutar un programa.

Ingresa powershell.exe como programa y escribir la ruta del script en "Argumentos":

Arduino

-File "C:\ruta\script.ps1"

Utilización de Extensiones de Comandos para Tareas de Administración en Bash y PowerShell

- Las **extensiones de comandos** en Bash y PowerShell permiten crear **comandos personalizados** que facilitan la automatización y administración de sistemas. Estas extensiones pueden ser **scripts en Bash** o **cmdlets en PowerShell**, y una vez configuradas correctamente, se pueden invocar como si fueran comandos del sistema.

Ventajas de Usar Extensiones de Comandos

- **Automatización de tareas:** Reduce el tiempo en tareas repetitivas.
- **Simplificación de la línea de comandos:** Se pueden usar comandos personalizados en lugar de scripts largos.
- **Flexibilidad y personalización:** Adaptables a necesidades específicas de administración del sistema.
- **Estandarización:** Mejora la eficiencia del equipo de TI con comandos comunes.

Interpretación de Scripts del Sistema y Adaptaciones entre Bash y PowerShell

Diferencias Clave entre Bash y PowerShell

Característica	Bash (Linux/Unix)	PowerShell (Windows)
Extensión de archivo	.sh	.ps1
Ejecución del script	bash script.sh o ./script.sh	powershell.exe -File script.ps1
Estilo de comandos	Comandos de Unix (ls, grep, awk)	Cmdlets (Get-Process, Get-Content)
Variables	variable="valor" (sin prefijo)	\$variable = "valor" (con \$)
Orientación	Basado en texto y flujos de datos	Orientado a objetos
Interacción con el sistema	Usa sed, awk, grep, cat para manipulación de datos	Usa cmdlets para administrar procesos, archivos y registros
Ejecución de otros scripts	./otro_script.sh	.\otro_script.ps1

Adaptación de Scripts entre PowerShell y Bash

- La adaptación de scripts entre PowerShell (Windows) y Bash (Linux/Unix) implica modificar comandos, sintaxis y estructuras de control para que funcionen correctamente en el otro sistema. Aunque ambos lenguajes tienen funcionalidades similares, sus diferencias en la sintaxis y enfoque pueden hacer que la conversión no sea siempre directa.

Consideraciones al Adaptar un Script

- **Sustituir comandos equivalentes**
 - **Ejemplo:** Get-ChildItem (PowerShell) : ls (Bash).
- **Modificar la sintaxis de variables y parámetros**
 - **PowerShell** usa \$variable, mientras que **Bash** usa variable sin prefijo.
- **Cambiar estructuras de control de flujo**
 - **Ejemplo:** if en PowerShell usa {}, mientras que en Bash usa then ... fi.
- **Ajustar rutas y convenciones del sistema operativo**
 - **Windows** usa C:\Ruta\Archivo, mientras que **Linux** usa /ruta/archivo.
- **Modificar la forma en que se manipulan archivos y procesos**
 - **Ejemplo:** Get-Content (PowerShell) : cat (Bash).

Scripts para la Gestión del Directorio Activo en PowerShell y Bash

- Los scripts en PowerShell (Windows) y Bash (Linux) permiten gestionar usuarios, grupos y permisos dentro del Directorio Activo (Active Directory, AD). Sin embargo, dado que PowerShell es nativo de Windows, tiene una integración más directa con AD, mientras que en Linux, se requiere el uso de herramientas LDAP.

Scripts para la Gestión del Directorio Activo en PowerShell y Bash

Gestión del Directorio Activo en PowerShell

- **Get-ADUser**: Para obtener información sobre los usuarios en el Directorio Activo.
- **New-ADUser**: Para crear un nuevo usuario en el Directorio Activo.
- **Set-ADUser**: Para modificar la información de un usuario existente en el Directorio Activo.
- **Get-ADGroup**: Para obtener información sobre los grupos en el Directorio Activo.
- **New-ADGroup**: Para crear un nuevo grupo en el Directorio Activo.
- **Set-ADGroup**: Para modificar la información de un grupo existente en el Directorio Activo.
- **Get-Acl**: Para obtener información sobre los permisos de acceso al registro de Windows.
- **Set-Acl**: Para modificar los permisos de acceso al registro de Windows.

Scripts para la Gestión del Directorio Activo en PowerShell y Bash

Administración de Usuarios en Bash (Linux)

- `cat /etc/passwd` : Para obtener información sobre los usuarios del sistema.
- `useradd` : Para crear un nuevo usuario en el sistema.
- `usermod` : Para modificar la información de un usuario existente.
- `passwd` : Para cambiar la contraseña de un usuario.
- `userdel` : Para eliminar un usuario del sistema.

Administración de Grupos en Bash (Linux)

- `cat /etc/group` : Para obtener información sobre los grupos del sistema.
- `groupadd` : Para crear un nuevo grupo en el sistema.
- `groupmod` : Para modificar la información de un grupo existente.
- `gpasswd` : Para gestionar la membresía de un grupo.
- `groupdel` : Para eliminar un grupo del sistema.

